

QCar2 Internship

Quang Thinh BUI

May 2026

QCar2: Flatness-based Mixed-Integer Obstacle Avoidance MPC

1 Introduction

For trajectory planning, the QCar2 is approximated by a kinematic single-track model, which provides a simpler representation than the full dynamic model. Let X and Y denote the Cartesian coordinates of the vehicle in the inertial frame, ψ the vehicle heading angle, and v_x the longitudinal velocity. Let δ be the front-wheel steering angle, a_x the longitudinal acceleration, and L_{wb} the wheelbase of the vehicle.

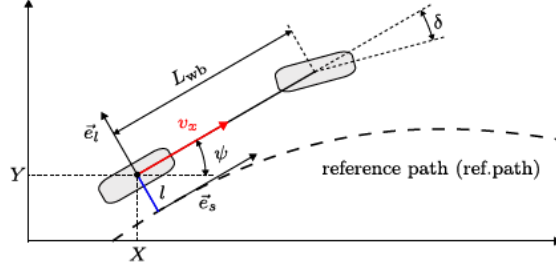


Figure 1: Kinematic single-track model.

The state and input vectors are defined as

$$\mathbf{x} = [X \quad Y \quad \psi \quad v_x]^\top, \quad \mathbf{u} = [\delta \quad a_x]^\top.$$

Under the pure-rolling and no-slip assumptions, the kinematic single-track model is given by

$$\dot{X} = v_x \cos \psi, \tag{1a}$$

$$\dot{Y} = v_x \sin \psi, \tag{1b}$$

$$\dot{\psi} = \frac{v_x}{L_{wb}} \tan \delta, \tag{1c}$$

$$\dot{v}_x = a_x. \tag{1d}$$

2 Obstacle Avoidance MPC

2.1 Differential Flatness of the Vehicle Model

The kinematic single-track model introduced previously is differentially flat when the Cartesian position of the vehicle is selected as the flat output. For the Cartesian formulation, the flat output is chosen as

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} X \\ Y \end{bmatrix}. \tag{2}$$

Its first derivative is obtained from the kinematic model as

$$\dot{\mathbf{y}} = \begin{bmatrix} \dot{y}_1 \\ \dot{y}_2 \end{bmatrix} = \begin{bmatrix} v_x \cos \psi \\ v_x \sin \psi \end{bmatrix}. \tag{3}$$

The corresponding flat state vector is then defined as

$$\mathbf{z} = \begin{bmatrix} y_1 \\ \dot{y}_1 \\ y_2 \\ \dot{y}_2 \end{bmatrix} = \begin{bmatrix} X \\ v_x \cos \psi \\ Y \\ v_x \sin \psi \end{bmatrix} = t(\mathbf{x}). \quad (4)$$

The inverse transformation from the flat state to the original vehicle state is given by

$$\mathbf{x} = \begin{bmatrix} X \\ Y \\ \psi \\ v_x \end{bmatrix} = \begin{bmatrix} z_1 \\ z_3 \\ \text{atan2}(z_4, z_2) \\ \sqrt{z_2^2 + z_4^2} \end{bmatrix} = t^{-1}(\mathbf{z}). \quad (5)$$

Using the virtual input vector

$$\boldsymbol{\nu} = \begin{bmatrix} \nu_1 \\ \nu_2 \end{bmatrix} = \begin{bmatrix} \ddot{y}_1 \\ \ddot{y}_2 \end{bmatrix}, \quad (6)$$

the flat dynamics become two decoupled double integrators:

$$\dot{\mathbf{z}} = \underbrace{\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}}_{\mathbf{A}_c} \mathbf{z} + \underbrace{\begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix}}_{\mathbf{B}_c} \boldsymbol{\nu}. \quad (7)$$

The virtual inputs are related to the physical inputs by differentiating the flat outputs twice:

$$\boldsymbol{\nu} = \begin{bmatrix} \nu_1 \\ \nu_2 \end{bmatrix} = \begin{bmatrix} a_x \cos \psi - \frac{v_x^2}{L_{wb}} \sin \psi \tan \delta \\ a_x \sin \psi + \frac{v_x^2}{L_{wb}} \cos \psi \tan \delta \end{bmatrix}. \quad (8)$$

Conversely, the physical input vector can be recovered from the virtual input as

$$\mathbf{u} = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = \begin{bmatrix} \delta \\ a_x \end{bmatrix} = \begin{bmatrix} \arctan \left(\frac{L_{wb}}{v_x^2} (-\nu_1 \sin \psi + \nu_2 \cos \psi) \right) \\ \nu_1 \cos \psi + \nu_2 \sin \psi \end{bmatrix}. \quad (9)$$

In practice, when v_x is close to zero, the denominator v_x^2 can be regularized by replacing it with $v_x^2 + \varepsilon$, where $\varepsilon > 0$ is a small constant.

2.2 OA-MPC Formulation

The flat representation allows the nonlinear vehicle model to be replaced by a linear prediction model in the flat coordinates. With sampling time T_s , the continuous-time flat model is discretized under a zero-order hold assumption as

$$\mathbf{z}_{k+1} = \underbrace{\begin{bmatrix} 1 & T_s & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & T_s \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{\mathbf{A}_d} \mathbf{z}_k + \underbrace{\begin{bmatrix} \frac{T_s^2}{2} & 0 \\ T_s & 0 \\ 0 & \frac{T_s^2}{2} \\ 0 & T_s \end{bmatrix}}_{\mathbf{B}_d} \boldsymbol{\nu}_k. \quad (10)$$

At each sampling instant, the OA-MPC solves the following optimization problem over the prediction horizon N_p :

$$\min_{\mathbf{z}, \mathbf{V}, s} \sum_{j=0}^{N_p-1} (\mathbf{e}_{k+j+1}^\top \mathbf{Q} \mathbf{e}_{k+j+1} + \boldsymbol{\nu}_{k+j}^\top \mathbf{R} \boldsymbol{\nu}_{k+j}) + r_s s \quad (11a)$$

$$\text{s.t. } \mathbf{z}_{k+j+1} = \mathbf{A}_d \mathbf{z}_{k+j} + \mathbf{B}_d \boldsymbol{\nu}_{k+j}, \quad (11b)$$

$$\boldsymbol{\nu}_{k+j} \in \mathcal{W}, \quad (11c)$$

$$\mathbf{z}_{k+j+1} \in \mathcal{Z}, \quad (11d)$$

$$s \geq 0, \quad (11e)$$

for $j = 0, \dots, N_p - 1$. Here, $\mathbf{z}$ and $\mathbf{v}$ denote the predicted flat state and virtual input sequences, respectively. The scalar s is a slack variable used to preserve feasibility, while r_s is chosen sufficiently large to strongly penalize constraint violation.

For the Cartesian flat output formulation, the tracking error is defined by

$$\mathbf{e}_{k+j} = \begin{bmatrix} e_X \\ e_Y \end{bmatrix}_{k+j} = \begin{bmatrix} X \\ Y \end{bmatrix}_{k+j} - \begin{bmatrix} X_{\text{ref}} \\ Y_{\text{ref}} \end{bmatrix}_{k+j}. \quad (12)$$

The physical input constraints are originally defined as

$$\mathcal{U} = \{\mathbf{u} : \mathbf{u}_{\min} \leq \mathbf{u} \leq \mathbf{u}_{\max}\}, \quad \mathbf{u} = \begin{bmatrix} \delta \\ a_x \end{bmatrix}. \quad (13)$$

Since the MPC is solved in the flat coordinates, these input constraints must be mapped to constraints on the virtual input $\boldsymbol{\nu}$. From the input transformation, one obtains

$$\underbrace{\begin{bmatrix} -\frac{L_{\text{wb}}}{v_x^2} \sin \psi & \frac{L_{\text{wb}}}{v_x^2} \cos \psi \\ \cos \psi & \sin \psi \end{bmatrix}}_{\mathbf{M}(\psi, v_x)} \begin{bmatrix} \nu_1 \\ \nu_2 \end{bmatrix} = \begin{bmatrix} \tan \delta \\ a_x \end{bmatrix} = \tilde{\mathbf{u}}. \quad (14)$$

Therefore, the flat input set is written as

$$\mathcal{W} = \{\boldsymbol{\nu} : \tilde{\mathbf{u}}_{\min} \leq \mathbf{M}(\psi, v_x) \boldsymbol{\nu} \leq \tilde{\mathbf{u}}_{\max}\}, \quad (15)$$

where

$$\tilde{\mathbf{u}}_{\min} = \begin{bmatrix} \tan \delta_{\min} \\ a_{x, \min} \end{bmatrix}, \quad \tilde{\mathbf{u}}_{\max} = \begin{bmatrix} \tan \delta_{\max} \\ a_{x, \max} \end{bmatrix}. \quad (16)$$

It should be noted that the flat input set $\mathcal{W}$ depends on the current vehicle state through ψ and v_x . Therefore, at each MPC sampling instant, the matrix $\mathbf{M}(\psi, v_x)$ is updated using the current measured or estimated state of the vehicle.

2.2.1 Obstacle Avoidance Constraints

Obstacle avoidance is formulated in the flat output space (y_1, y_2) . For the Cartesian case, this corresponds directly to the global position (X, Y) . A convex polygonal obstacle region $\mathcal{Q}$ with n_e edges is represented by a set of half-space inequalities:

$$\mathcal{Q} = \left\{ \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} : \mathbf{G} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} \leq \mathbf{f} \right\}, \quad (17)$$

where $\mathbf{G} \in \mathbb{R}^{n_e \times 2}$ and $\mathbf{f} \in \mathbb{R}^{n_e}$.

To avoid the obstacle, the predicted vehicle position must lie outside $\mathcal{Q}$. This condition is non-convex because the vehicle must violate at least one of the obstacle half-space inequalities. The Big-M method is used to encode this disjunctive condition with binary variables $\boldsymbol{\alpha}_{k+j} \in \{0, 1\}^{n_e}$:

$$-\mathbf{G} \begin{bmatrix} y_{1, k+j} \\ y_{2, k+j} \end{bmatrix} \leq -\mathbf{f} - (\gamma - s) \mathbf{1} + M(\mathbf{1} - \boldsymbol{\alpha}_{k+j}), \quad (18a)$$

$$\mathbf{1}^\top \boldsymbol{\alpha}_{k+j} \geq 1, \quad (18b)$$

$$\boldsymbol{\alpha}_{k+j} \in \{0, 1\}^{n_e}, \quad (18c)$$

$$s \geq 0. \quad (18d)$$

Here, M is a sufficiently large positive constant, γ is a safety buffer distance, and s is a scalar slack variable. The constraint $\mathbf{1}^\top \boldsymbol{\alpha}_{k+j} \geq 1$ enforces that at least one edge constraint of the obstacle is violated. Therefore, the predicted vehicle position is forced to remain outside the obstacle region. The safety buffer γ inflates the obstacle region to compensate for the finite size of the vehicle, since the ego vehicle is represented as a point in the flat output space.

For multiple obstacles, the same set of constraints is added for each relevant obstacle and for each prediction step. As a result, the OA-MPC problem becomes a mixed-integer quadratic programming problem.

After solving the OA-MPC problem, the first optimal virtual input $\boldsymbol{\nu}_k^*$ is converted back to the physical input using the current vehicle state $(\psi_k, v_{x,k})$:

$$\mathbf{u}_k^* = \begin{bmatrix} \delta_k^* \\ a_{x,k}^* \end{bmatrix} = \begin{bmatrix} \arctan \left(\frac{L_{\text{wb}}}{v_{x,k}^2} \left(-\nu_{1,k}^* \sin \psi_k + \nu_{2,k}^* \cos \psi_k \right) \right) \\ \nu_{1,k}^* \cos \psi_k + \nu_{2,k}^* \sin \psi_k \end{bmatrix}. \quad (19)$$

Only this first control action is applied to the vehicle. At the next sampling instant, the current state is measured again, the flat input set $\mathcal{W}$ is updated, and the OA-MPC problem is solved again in a receding-horizon manner.